

条款19. 对于共享资源使用std::shared_ptr

①核心作用: 引用计数管理同一对象

②基础用法:

```
#include <memory>
```

```
std::shared_ptr<int> p1(new int(10));
```

```
auto p2 = std::make_shared<int>(20); //更推荐
```

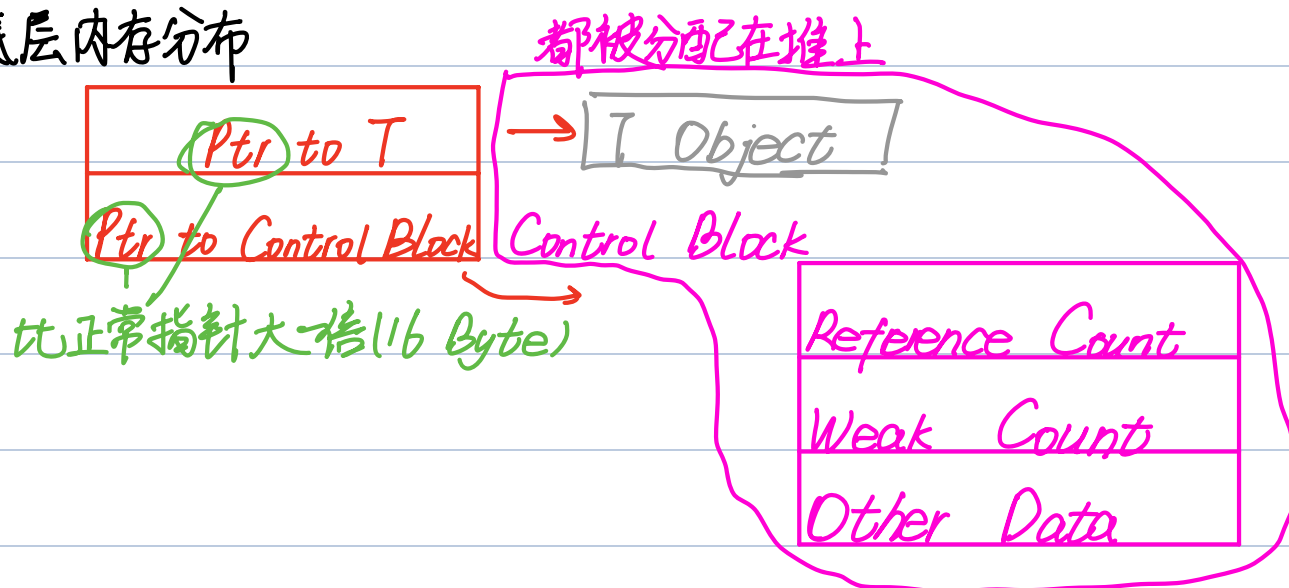
```
auto p3 = p2, p4 = p1; //支持拷贝
```

```
p1.get(); //获取裸指针
```

```
p1.use_count(); //当前引用计数
```

```
p1.unique(); //是否只有一个持有者
```

③底层内存分布



自定义删除器(deleter)不会增加shared_ptr的大小

而删除器是unique_ptr类型的一部分, share_ptr不是

```
auto loggingD = [](Widget* pw){ makeLogEntry(pw);  
                                delete pw;}
```

```
std::unique_ptr<Widget, decltype(loggingD)> upw(  
    new Widget, loggingD);
```

```
std::shared_ptr<Widget> spw(new Widget, loggingD);
```

④ shared_ptr的性能问题

- a. 大小是原始指针2倍 b. 引用计数内存必须动态分配
- c. 引用计数的增减必须是原子的

⑤ control block的生成时机

- a. 使用std::make_shared b. 通过unique_ptr构造
- c. 构造shared_ptr传入一个裸指针

⑥ shared_ptr存在的问题

T object 与 Control Block 关联在一起

但如果2个Control Block与1个T object关联

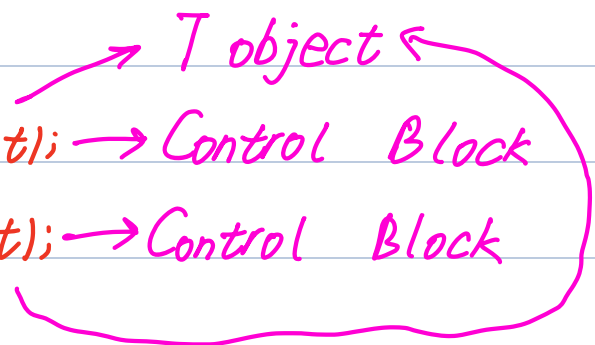
大概率会出现被多次释放的风险

```
auto pt = new Test;
```

```
std::shared_ptr<Test> spt1(pt);
```

```
std::shared_ptr<Test> spt2(pt);
```

太复杂




`std::shared_ptr<Test> spt1(new int(10));`
`std::shared_ptr<Test> spt2(spt1);`

⑦ 使用 `this` 指针作为 `std::shared_ptr` 构造函数实参
详情看 `Widget` 类部分

⑧ `shared_ptr` 不支持数组

`shared_ptr<T[]>` 没有这种写法